# Capacidades e arquitetura do Vladmir

Leitura técnica do branch main em 14/10/2026. Este inventário descreve o que o código propõe e
implementa. Não é certificação, resultado de teste de hardware nem garantia de operação em todas as distribuições.

## Resumo executivo

Vladmir é composto por um cliente Qt para Linux, um backend Python que usa Ollama local, serviços locais para voz/visão e dados em SQLite/arquivos privados, além de clientes Android/Wear. O desenho favorece inferência local e integrações opcionais sob demanda. O repositório inclui limites e confirmações em várias ações sensíveis, mas integração local ainda depende muito de configuração, versão dos serviços e estado dos dispositivos.

| Área | Estado no código | Dependências e limite principal |
| --- | --- | --- |
| Chat de desktop | Implementado | Ollama e modelo instalado; contexto finito |
| Perfis de chat, engenharia e ensino | Implementado | Os perfis não criam modelos nem capacidade ausentes |
| Memória local compartilhada | Implementada | Persistência local; memória/contexto limitados |
| Biblioteca de conhecimento | Implementada | Extração parcial; quotas e formatos específicos |
| Voz e síntese | Implementadas | Whisper, Piper, FFmpeg e rota de áudio do sistema |
| Chamada no desktop/mobile | Implementada por turnos | Não é áudio full-duplex em tempo real |
| Visão e câmera | Implementadas | Captura de frame; câmera/modelo/FFmpeg precisam funcionar |
| Busca/leitura web | Implementadas como integração nativa | Internet e provedor; páginas podem falhar ou mudar |
| GitHub | Leitura e edição explícita | gh, rede e permissão da conta; limites de conteúdo |
| Plugins | Integrações nativas | Sem marketplace genérico de extensões |
| JetBrains MCP | Cliente local experimental | Servidor da IDE precisa ser iniciado e compatível |
| Agenda e lembretes | Implementados localmente | Sem sincronização de calendário externo |
| Email | Preparação de rascunho | Envio final ocorre no cliente de email |
| Sentinel | Varredura local integrada | ClamAV/assinaturas; cobertura limitada ao caminho selecionado |
| Instalação de software | Solicitação limitada e confirmada | Pacotes/gerenciadores permitidos; backup é inventário |
| Autoevolução de código | Candidatos isolados | Revisão/promoção não automáticas |
| Treino de adaptador | Fluxo experimental protegido | Dataset, autorização, GPU e avaliação necessários |
| Android | Código Kotlin/Compose | SDK, build, servidor, TLS, pareamento e permissões |
| Companion/Wear | Código nativo e protocolo | Pareamento e comportamento de segundo plano precisam validação física |

1. Componentes

Desktop e núcleo

src/mark/desktop.py implementa a interface PyQt. Os módulos desktop_* organizam diálogos de visão, memória, biblioteca, plugins, JetBrains, Sentinel, evolução e outras ferramentas. brain*, core/ e cognition/ lidam com roteamento de prompts, provedores, ferramentas, contexto e funções cognitivas. A chamada é iniciada no mesmo app desktop.

O usuário seleciona os perfis Geral, Engenheiro e Professor. Existem conversas especializadas e memória compartilhada, mas retenção é limitada pelos bancos locais, contexto carregado e política do módulo. O seletor “instantâneo/médio/alto” escolhe opções de geração e contexto; não treina o modelo nem garante aumento mensurável de qualidade.

Inferência local e eficiência

O caminho padrão usa HTTP no Ollama local, sem pesos embutidos. Perfis de modelo vêm de models e ollama em config.json. keep_alive: 0 solicita descarregamento depois da chamada para liberar VRAM; isso pode elevar latência de próxima resposta. O desempenho depende do modelo, quantização, backend CUDA/CPU, concorrência e RAM/VRAM disponíveis.

Os modelos nomeados no exemplo são sugestões, não dependências universais. Sem tag correspondente no Ollama, a chamada falha. O projeto não converte automaticamente modelos em uma variante “Heretic”, não fornece pesos proprietários e não pode inferir capacidades apenas pelo nome.

2. Chat, contexto e especializações

* Geral: chat primário; aceita anexos limitados, opções de raciocínio e ferramentas disponíveis no backend.
* Engenheiro: roteia para perfil de programação e oferece espaço de trabalho, missões, leitura de código, verificação local e integrações configuradas.
* Professor: organiza materiais e estudo, com memória compartilhada e referência opcional à biblioteca.
* Contexto: conversa persistida não significa que todo o histórico é enviado em cada geração. Há limite de mensagens e janela do modelo.
* Ferramentas: o modelo não obtém automaticamente acesso irrestrito ao computador. A aplicação decide ferramentas, permissões e confirmações.

3. Biblioteca e extração

cognition/library_extract.py limita arquivo de entrada a 64 MiB, conteúdo expandido de arquivo compactado a 32 MiB e extração a 2 milhões de caracteres. O chat possui limites separados (5 arquivos; 6 MiB cada; 8 MiB total; 400 mil caracteres de texto extraído).

Há leitores para PDF, DOCX, EPUB, HTML, CSV, JSON, YAML, XML, muitos formatos de código e formatos adicionais por parser universal. A extração depende de ferramentas como pdftotext, Tesseract/OCR, ffmpeg/ffprobe e Whisper quando aplicável. A leitura pode se limitar a páginas/trechos ou metadados. Fórmulas de planilhas, imagens, arquivos compactados, documentos protegidos e binários não são necessariamente interpretados como uma pessoa faria. Importar para a biblioteca não ajusta os pesos do modelo.

4. Entrada/saída de voz

audio.py captura fluxo Pulse, mono, 16 kHz por padrão, calibra limiar e detecta enunciados. Whisper transcreve; tts.py sintetiza com Piper e pode recorrer a espeak-ng. A seleção de dispositivo usa configuração Pulse/PipeWire. Falta de modelos ou dispositivos resulta em falha ou fallback, não em substituição perfeita.

A chamada móvel/desktop envia turnos de áudio à API. Não há garantia de fluxo bidirecional continuamente aberto, interrupção em tempo real em todas as rotas, baixa latência constante ou coexistência perfeita com áudio Bluetooth de alta qualidade.

5. Visão

vision.py captura um frame via FFmpeg, verifica que os bytes representam imagem e passa a imagem ao perfil compatível com visão. A fonte pode ser USB ou RTSP(S). Há módulos de fluxo ao vivo/foco/qualidade; a cadência é limitada. O caminho não equivale a gravação contínua, reconhecimento de todos os objetos, rastreamento persistente ou substituição de sensor especializado.

Dependências típicas: dispositivo visível para Linux, permissão de câmera, FFmpeg, URL autenticada válida e modelo multimodal. Uma URL válida ou uma câmera listada não prova que o stream esteja acessível.

6. Web, integrações e plugins

O plugin web consulta buscador e páginas públicas. O código filtra destinos não públicos para reduzir SSRF; essa proteção também significa que não é navegador de intranet nem acesso sem limites à rede.

O plugin github lê projetos acessíveis via API/gh; edição ocorre por ação explícita e confirma a gravação. A conta precisa estar autenticada e ter escopo adequado. JetBrains MCP aceita servidor HTTP em localhost, lista ferramentas e pede confirmação antes da execução. As operações possíveis dependem das ferramentas expostas pela IDE.

O registro de plugins (plugins.py) é uma lista pequena de recursos nativos. O próprio diálogo informa que plugins arbitrários externos não são executados. Não há sandbox universal para extensões baixadas de um marketplace.

7. Recursos auxiliares

* Agenda: SQLite local, inclui tarefas, data/hora e conclusão; lembretes são acionados pela aplicação e não sincronizam calendário externo.
* Email: EmailComposerDialog monta mailto:; usuário revisa e envia no aplicativo padrão.
* God’s Eye View: ferramenta geográfica local sob demanda; dados e visualização dependem dos provedores selecionados e da rede.
* Sentinel: usa ClamAV no caminho escolhido e registra relatório local. Não é suíte EDR nem validação de todo o sistema. VirusTotal não recebe hash por padrão.
* Pacotes: package_install.py restringe gerenciadores/origens e cria inventário antes da instalação autorizada. Inventário não reverte todos os efeitos colaterais de pacotes.

8. API móvel e autenticação

mobile_api.py fornece endpoints de saúde, modelos, agenda, histórico, status de pareamento, sessão, chat completions, voz e telemetria do Wear. Por padrão a API é local. Autenticação vincula sessão/dispositivo; ações críticas ficam bloqueadas para cliente não proprietário. Corpos, texto e anexos possuem limites explícitos. Uma requisição concorrente pode receber HTTP 429.

O caminho remoto exige configuração consciente de TLS/túnel, endereços e autorização. Não há endpoint pessoal embutido. A conectividade física deve ser comprovada com o app do servidor ativo, celular disponível, certificados/endereço válidos, pareamento e regras de rede apropriadas.

9. Android, Companion e Wear OS

O cliente Android contém chat, captura de foto, voz, anexos e ARCore opcional. Dispositivo sem ARCore suportado ou sem Play Services for AR não executa o modo AR. O mapa AR atual dura apenas enquanto a tela permanece ativa; não persiste entre sessões.

O Companion pode retransmitir notificações selecionadas, inclusive notificação de WhatsApp com prévia limitada, quando o usuário concede Notification Listener Access. O protocolo Bluetooth autenticado depende de pareamento e serviço no Android. O diagnóstico da interface registra que reconexão automática ainda não está habilitada; economia de bateria do fabricante pode suspender o processo. O Wear enumera sensores disponíveis, mas disponibilidade e permissões variam por modelo. Não se deve tratar leitura de batimentos ou orientação como medição médica.

O código-fonte não comprova que APKs tenham sido compilados, instalados, homologados ou que a conexão fique estável com tela desligada. Isso exige build/assinatura e ensaios em aparelhos e versões concretas.

10. Privacidade e segurança

Dados são armazenados sob pastas locais do usuário. O projeto recomenda config 0600, API em loopback e túnel autenticado para acesso externo. Web, GitHub, Honcho e VirusTotal envolvem envio de dados a terceiros quando usados; Honcho e VirusTotal ficam desligados por padrão na configuração de exemplo. Revise política de cada serviço antes de habilitar.

Os limites de app não substituem atualização do sistema, antivírus completo, backup, controle de permissões ou revisão de código. Relate vulnerabilidades em privado conforme SECURITY.md.

## 11. Autoevolução e estado de maturidade

evolution/engine.py executa experimento isolado, coleta comparação e grava relatórios. A decisão de candidato continua marcada como REVIEW_REQUIRED, sem autorização de promoção. A aplicação ao runtime é uma ação de manutenção distinta. O painel tem start/stop e rotina systemd opt-in; a rotina não deve ser presumida ativa em instalação nova.

automatic_training.py requer contrato com autorização explícita, dataset aprovado pelo proprietário, arquivos com hash, holdout e parâmetros limitados. A medição de energia/qualidade/VRAM pode faltar; sem medição válida, o candidato não deve ser considerado eficiente. O processo não atualiza pesos ativos por conta própria.

## 12. Como confirmar capacidade numa instalação

1. Registre o commit instalado e o sistema operacional.
2. Rode o --check e guarde apenas saída sem segredos.
3. Confirme Ollama/modelos locais.
4. Teste cada recurso por separado com dispositivo/serviço disponível.
5. Para API, valide /health, /v1/models e autenticação apropriada sem expor token.
6. Para câmera/áudio/Bluetooth, faça teste físico e verifique estado do dispositivo.
7. Anote sucesso, erro, latência, dependências e escopo do teste.

## 13. Estrutura, testes e automação de integração

O repositório tem diretórios de código-fonte (src/mark/), testes (tests/), scripts de operação (scripts/) e projetos Android/Wear separados. Há 37 arquivos com padrão tests/test_*.py, usando unittest, além de scripts de teste unitário e smoke de integração. A suíte automatizada existe, mas seu número de arquivos não é uma métrica de cobertura.

No commit d9b3358, não havia arquivos de workflow em .github/workflows/, nem configuração/relatório publicado de coverage. Portanto, o estado correto é “testes presentes; CI e percentual de cobertura ainda não publicados”, e não “sem testes” ou “cobertura zero”. O smoke de integração depende de Ollama, modelos e recursos locais; uma execução de unidade não demonstra que a integração completa ou hardware está funcionando.

O README traz o comando da suíte local. Esta documentação não declara que os testes foram executados durante a inspeção.

## 14. Referências do código

src/mark/desktop.py, src/mark/brain*.py, src/mark/core/, src/mark/cognition/, src/mark/mobile_api.py, src/mark/attachment_limits.py, src/mark/plugins.py, src/mark/github.py, src/mark/jetbrains_mcp.py, src/mark/package_install.py, src/mark/sentinel.py, src/mark/vision*.py, src/mark/audio.py, src/mark/tts.py, src/mark/agenda.py, src/mark/evolution/, mobile-android/, vladmir-companion/, vladmir-wear/.